UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2025-02, page [2]

Unofficial model answer:

a) The primary technologies are: zeroth generation — mechanical relays (electromechanical relays); first generation — vacuum tubes; second generation — transistors. b) Two fundamental components of an integrated circuit are the transistor and the resistor (other passive components such as capacitors and diodes are also fabricated on the chip, but the transistor and resistor are the standard fundamental building blocks).

Marking points:

- *1 mark: zeroth generation primary technology = mechanical/electromechanical relays*
- *1 mark: first generation primary technology = vacuum tubes*
- *1 mark: second generation primary technology = transistors*
- *1 mark: one fundamental IC component (e.g., transistor)*
- *1 mark: second fundamental IC component (e.g., resistor, capacitor, or diode)*

Model confidence: high.

Model note: Some textbooks label the pre-vacuum-tube era as 'mechanical' or 'electromechanical' rather than 'zeroth generation'; either term is accepted. For part (b), the question asks for two components; transistor and resistor are the most commonly cited, but capacitors/diodes are also valid IC elements.

QUESTION 2 (5 marks)

Source: CSC429-2018-01, page [2]

Unofficial model answer:

a) The personal computer (the Altair 8800, 1975, cited in Figure 1) belongs to the FOURTH computer generation, the microprocessor/VLSI era (roughly 1971 onward). This generation is defined by the invention of the microprocessor — the Intel 4004 (1971) and later 8080 — which put a complete CPU on a single chip and made cheap desktop machines like the Altair possible.

b) One other major discovery of the fourth generation is Very Large Scale Integration (VLSI): packing thousands to millions of transistors onto one silicon chip. VLSI (together with the microprocessor it enabled) produced dramatic reductions in size, cost and power per function, and gave rise to microcomputers, inexpensive semiconductor memory, and the graphical user interface/personal-computing revolution. (Acceptable alternatives: the microprocessor itself if not used in (a), or large-scale semiconductor memory replacing core memory.)

c) Laptop and hand-held devices are feasible now because of continued scaling of VLSI into ULSI and system-on-chip (SoC) design: far more transistors per chip at lower cost, lower power and higher clock speed, plus low-power CMOS, high-density memory, flat-panel displays and wireless/battery technology. Compared with earlier generations (vacuum tubes, then discrete transistors, then SSI/MSI integrated circuits), which were bulky, hot, power-hungry and expensive, modern devices integrate CPU, memory and I/O on a few chips, so the same computing power that once filled a room now fits in a hand-held device.

Marking points:

- *1 mark (a): Correctly identifies the fourth generation (microprocessor/VLSI era, ~1971 onward) as the generation in which the personal computer appeared.*
- *2 marks (b): Names one valid major discovery of that generation (e.g., VLSI, or the microprocessor) — 1 mark; gives a correct, relevant discussion of its significance/impact — 1 mark.*
- *2 marks (c): Explains feasibility via VLSI/ULSI/SoC scaling and low-power, high-density integration — 1 mark;*

© Hak Cipta Universiti Teknologi MARA

contrasts with previous generations (tubes/transistors/SSI-MSI: bulky, costly, power-hungry) to justify the advance — 1 mark.

Model confidence: high.

Model note: Figure 1 is only a text snippet (Altair 8800, 1975); no diagram content is needed. Generation numbering follows the standard CSC429 convention (1st: vacuum tubes; 2nd: transistors; 3rd: ICs; 4th: microprocessors/VLSI; 5th: AI/ULSI). This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 3 (5 marks)

Source: CSC429-2024-07, page [3]

Unofficial model answer:

a) Sign-and-magnitude subtraction $246_s - 1A_{16}$. Interpret the operands as bytes: 246_s means 24_6 (base 6) = 2×6+4 = 16_{10} = 0001 0000$_2$; $1A_{16}$ = 26_{10} = 0001 1010$_2$. In sign-and-magnitude byte form, +16 = 0001 0000 and +26 = 0001 1010 (sign bit 0, 7-bit magnitude). Since 16 < 26, the result is negative: magnitude = 26 − 16 = 10_{10} = 000 1010$_2$. Attach sign bit 1: 1000 1010$_2$ = 8A$_{16}$. Workflow: convert both to binary bytes → compare magnitudes → subtract smaller from larger → set sign bit to 1 because the larger operand is negative. Answer: 10001010$_2$ (8A$_{16}$), i.e. -10_{10}.

b) $-3E_{16} + 45_s$ using 1's and 2's complement. $3E_{16}$ = 0011 1110$_2$ = 62_{10}, so $-3E_{16} = -62_{10}$. 45_s = 4×5+5 = 25_{10} = 0001 1001$_2$. 1's complement: +62 = 0011 1110; −62 = 1100 0001 (invert). Add +25: 1100 0001 + 0001 1001 = 1101 1010. No end-around carry; result is negative in 1's complement: magnitude = invert(1101 1010) = 0010 0101 = 37_{10}, so result = -37_{10} = 1101 1010$_2$ (DA$_{16}$). 2's complement: −62 = 1100 0010 (invert +1). Add +25: 1100 0010 + 0001 1001 = 1101 1011. Sign bit 1, so negative; magnitude = 2's complement back = 0010 0101 = 37_{10}, giving -37_{10} = 1101 1011$_2$ (DB$_{16}$). Workflow: convert operands to bytes → form negative operand in 1's/2's complement → binary add → interpret sign and recover magnitude. Answers: 1's complement 11011010$_2$ (DA$_{16}$); 2's complement 11011011$_2$ (DB$_{16}$), both representing -37_{10}.

Marking points:

- *1 mark: correct conversion of 246_s (=16_{10}) and $1A_{16}$ (=26_{10}) to byte binary*
- *1 mark: correct sign-and-magnitude subtraction workflow and sign determination (negative, magnitude 10)*
- *1 mark: correct byte answer 10001010$_2$ / 8A$_{16}$ for part (a)*
- *1 mark: correct 1's complement working and result 11011010$_2$ (DA$_{16}$) for part (b)*
- *1 mark: correct 2's complement working and result 11011011$_2$ (DB$_{16}$) for part (b)*

Model confidence: medium.

Model note: Subscript notation is ambiguous in the extracted text: '246s' and '45s' are read as base-6 numerals (subscript 6 rendered as 's'), and '1Ai6'/'3Ei6' as hexadecimal (subscript 16 rendered as 'i6'). If the intended bases differ, numeric values and final bytes change. No figure/table supplied; answer based on extracted text only. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2018-12, page [4]

Unofficial model answer:

a) Three interfaces to external devices are: (1) SCSI (Small Computer System Interface), a parallel interface used to connect disks and other peripherals; (2) USB (Universal Serial Bus), a serial interface for connecting a wide range of external devices; and (3) FireWire (IEEE 1394), a high-speed serial interface often used for external storage and multimedia devices. Other acceptable examples include IDE/ATA, SATA, and Thunderbolt.

b) A 'hot connection' (hot plugging) is the ability to connect or disconnect a device to or from a running system without powering down the computer or interrupting operation. It requires hardware and software support: the interface and device must be designed to be safely inserted/removed while powered, and the operating system

© Hak Cipta Universiti Teknologi MARA

must detect the change, load or unload the appropriate device driver, and reconfigure the system dynamically. Hot connection improves availability, convenience, and expandability, and is common with interfaces such as USB and FireWire. It also introduces concerns such as electrical protection (surge/ESD), safe removal to avoid data loss, and resource reallocation.

Marking points:

• 1 mark: correct identification of one external device interface (e.g., SCSI, USB, FireWire) with brief description

• 1 mark: correct identification of a second external device interface

• 1 mark: correct identification of a third external device interface

• 1 mark: definition of hot connection as connecting/disconnecting a device while the system is powered and running

• 1 mark: explanation of required support (hardware/software, OS detection, driver loading, safe removal) or benefits/concerns

Model confidence: high.

Model note: The question does not specify a particular textbook; answers use standard computer organization terminology. Alternative valid interfaces (e.g., IDE, SATA, Thunderbolt) should be accepted. No figure or table was provided.

QUESTION 5 (10 marks)

Source: CSC429-2025-07, page [4]

Unofficial model answer:

a) SISD architecture diagram: The SISD (Single Instruction, Single Data) architecture consists of a Control Unit (CU), a Processing Unit (PU), and a Memory Unit (MU). The CU fetches instructions from memory via the instruction stream (IS) and issues control signals to the PU. The PU executes the instruction and accesses data from memory via the data stream (DS). A typical block diagram: MU is connected to CU and PU; CU sends control signals to PU; IS flows from MU to CU; DS flows between MU and PU. (Diagram should show: MU ↔ CU (IS), MU ↔ PU (DS), CU → PU (control)).

b) Two examples of shared-memory MIMD: (1) Multicore processors (e.g., Intel Core i7, AMD Ryzen) where multiple cores share a common main memory. (2) Symmetric Multiprocessor (SMP) systems, such as a server with multiple CPUs sharing the same RAM.

c) Difference between virus attack on shared-memory MIMD vs distributed-memory MIMD: In shared-memory MIMD, all processors access a single global memory, so a virus can spread rapidly by infecting shared data or code in the common memory, potentially compromising all processors and the entire system. In distributed-memory MIMD, each processor has its own private memory and communicates via message passing; a virus would be confined to the infected node's local memory and would need to explicitly send malicious messages to other nodes to propagate, making containment easier and limiting the spread.

d) One benefit of distributed-memory MIMD: Scalability. Example: A cluster of commodity computers (e.g., Beowulf cluster) can be easily expanded by adding more nodes, each with its own memory, to increase processing power without the bottleneck of shared memory contention. This allows cost-effective scaling for large-scale scientific simulations.

Marking points:

• 1 mark: Correctly draws SISD diagram with CU, PU, MU labeled.

• 1 mark: Shows instruction stream (IS) from MU to CU and data stream (DS) between MU and PU.

• 1 mark: Shows control signals from CU to PU.

• 1 mark: Lists one correct example of shared-memory MIMD (e.g., multicore processor).

• 1 mark: Lists a second correct example (e.g., SMP server).

• 1 mark: Explains that in shared-memory MIMD, virus can spread via common memory to all processors.

• 1 mark: Explains that in distributed-memory MIMD, virus is confined to local node and requires message passing to spread.

• 1 mark: States one benefit of distributed-memory MIMD (e.g., scalability).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Provides a relevant example (e.g., cluster computing).

• 1 mark: Clearly contrasts the two scenarios or provides a coherent discussion.

Model confidence: medium.

Model note: The SISD diagram is described textually because the original figure cannot be rendered; the answer assumes a standard block diagram. The marking scheme is unofficial and based on typical exam expectations. The virus discussion is open-ended; other valid points (e.g., ease of recovery, performance impact) could be accepted.

QUESTION 6 (10 marks)

Source: CSC429-2023-02, page [5]

Unofficial model answer:

a) i) MIMD with shared memory: Draw several CUs (CU1, CU2, ... CUn), each with its own Instruction Stream (IS1, IS2, ... ISn) entering it from above. Each CU drives its own PU (PU1...PUn). All PUs connect through a single common interconnection network (e.g. a bus) to one shared MU (or shared memory block). The Data Streams (DS) are drawn as double-headed arrows between each PU and the shared MU via the interconnection network, because data flows both ways (read and write). Label the shared memory 'MU (shared)' and label each IS and DS arrow. Key point: one memory space, multiple independent instruction streams, multiple data streams.

a) ii) MIMD with distributed memory: Draw n independent nodes. Each node contains its own CU, its own PU and its own local MU (CU1+PU1+MU1, CU2+PU2+MU2, ... CUn+PUn+MUn). Each CU receives its own Instruction Stream (IS1...ISn). Each PU has a Data Stream (DS) to its own local MU. The nodes are joined by an interconnection network (e.g. a message-passing network) drawn between the PUs/nodes. Key point: each processor has a private memory; no single shared MU; processors communicate by messages over the network.

b) i) MIMD with shared memory example: a symmetric multiprocessor (SMP) server/workstation or a multicore CPU where several cores share one main memory (e.g. a dual/quad-core PC with a common RAM and cache-coherence protocol).

b) ii) MIMD with distributed memory example: a cluster of networked computers/Beowulf cluster, or a massively parallel processor such as a distributed-memory supercomputer where each node has its own memory and nodes exchange messages (e.g. via MPI).

c) Advantage of MIMD with distributed memory: scalability. Because each processor has its own private local memory, adding more nodes adds both processing power and memory bandwidth without all processors contending for one shared bus/memory; this avoids the shared-memory bottleneck and allows the system to grow to very large numbers of processors (and each processor accesses its local memory quickly).

Marking points:

• a) i) 3 marks: correct set of multiple CUs each with its own labelled IS (1), multiple PUs each with its own DS (1), all connected to one shared MU via an interconnection network with IS/DS clearly labelled (1).

• a) ii) 3 marks: n nodes each with own CU, PU and local MU (1), separate labelled IS per CU and DS between PU and local MU (1), interconnection network linking the nodes shown and labelled (1).

• b) i) 1 mark: correct shared-memory MIMD example (e.g. SMP/multicore with common RAM).

• b) ii) 1 mark: correct distributed-memory MIMD example (e.g. cluster/distributed-memory supercomputer).

• c) 2 marks: states scalability/no shared-memory bottleneck (1) with a brief justification that private local memory avoids contention and allows growth (1).

Model confidence: high.

Model note: Figure 3 is supplied only as a caption/crop description (SISD with CU, PU, MU and IS/DS arrows); the diagrams in (a) are described in words because a drawing cannot be rendered in JSON. Marking is an unofficial model allocation, not an official UTM scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION 7 (10 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

In a typical computer system, registers are small, fast storage locations inside the CPU used to hold data, addresses, and control information during program execution. The main registers and their functions are: (1) Program Counter (PC) / Instruction Pointer: holds the address of the next instruction to be fetched; it is automatically incremented or updated after each fetch so the CPU knows where to get the next instruction. (2) Instruction Register (IR): holds the instruction currently being fetched/decoded/executed; its contents are decoded by the control unit to determine the operation to perform. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed (read or written); it connects the CPU to the memory address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it acts as a buffer between the CPU and memory data bus. (5) Accumulator (AC): a general-purpose register used to hold operands and results of arithmetic and logic operations; in simple architectures it is implicitly used by many instructions. (6) General-Purpose Registers (GPRs): a set of registers (e.g., R0–R7) used for temporary storage of data, operands, intermediate results, and addresses; they reduce memory accesses and support flexible programming. (7) Status/Flag Register (Program Status Word, PSW): contains condition flags (zero, carry, sign, overflow) and control bits (interrupt enable, mode) that record the outcome of operations and the CPU state. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for subroutine calls, returns, and interrupt handling. (9) Index Register: holds an offset used in address calculation for indexed addressing, facilitating array and table access. (10) Instruction Decoder/Control registers (e.g., control unit registers): hold decoded control signals and sequencing information; sometimes a separate instruction address register or temporary registers are used in the datapath. In summary, registers provide the CPU with fast temporary storage for instructions, data, addresses, and status information, enabling the fetch-decode-execute cycle.

Marking points:

- *1 mark: Program Counter (PC) – holds address of next instruction; updated after fetch.*
- *1 mark: Instruction Register (IR) – holds current instruction for decoding/execution.*
- *1 mark: Memory Address Register (MAR) – holds address of memory location to be accessed.*
- *1 mark: Memory Data Register (MDR/MBR) – buffers data transferred to/from memory.*
- *1 mark: Accumulator (AC) – holds operands and results of arithmetic/logic operations.*
- *1 mark: General-Purpose Registers – temporary storage for data, operands, intermediate results.*
- *1 mark: Status/Flag Register (PSW) – holds condition flags and CPU state bits.*
- *1 mark: Stack Pointer (SP) – points to top of stack for subroutine/interrupt handling.*
- *1 mark: Index Register – holds offset for indexed addressing/address calculation.*
- *1 mark: Any other relevant register (e.g., instruction decoder/control register, temporary register) with correct function, or overall clarity and completeness.*

Model confidence: high.

Model note: The question does not specify a particular architecture; the answer assumes a generic/typical CPU with common registers. Some architectures may name or organize registers differently (e.g., no dedicated accumulator, multiple MAR/MDR). If the original paper included a figure or register list, the answer should be matched to that list; here only the question text is available.

QUESTION 8 (10 marks)

Source: CSC429-2015-12, page [4]

Unofficial model answer:

The four access methods based on the hardware implementation of a storage device are: (1) Sequential access — data is read or written in a fixed linear order, one record after another; the access time depends on the position of the desired record relative to the current position. Example: magnetic tape drive. (2) Direct

© Hak Cipta Universiti Teknologi MARA

access — data is organised into blocks or records that can be accessed in any order, but the device still uses a shared read/write mechanism that must move to the target block; access time depends on the block location. Example: hard disk drive (HDD). (3) Random access — any storage location can be accessed directly in approximately the same amount of time, independent of the previous access; each location has a unique address and the hardware can select it immediately. Example: main memory (RAM) or solid-state drive (SSD). (4) Associative access — data is accessed by content rather than by address; the hardware compares a search key against stored tags in parallel and returns matching entries. Example: cache memory (associative cache) or content-addressable memory (CAM).

Marking points:

• 1 mark: Correctly naming sequential access and giving a valid example (e.g., magnetic tape).

• 1 mark: Correctly describing sequential access (linear order, access time depends on position).

• 1 mark: Correctly naming direct access and giving a valid example (e.g., hard disk drive).

• 1 mark: Correctly describing direct access (blocks/records accessed in any order but with a shared mechanism; access time depends on block location).

• 1 mark: Correctly naming random access and giving a valid example (e.g., main memory/RAM or SSD).

• 1 mark: Correctly describing random access (any location accessed in approximately equal time, independent of previous access).

• 1 mark: Correctly naming associative access and giving a valid example (e.g., cache memory or CAM).

• 1 mark: Correctly describing associative access (access by content, parallel comparison of tags).

• 1 mark: Clear organisation and use of correct terminology across all four methods.

• 1 mark: All four methods and examples are distinct and appropriate to hardware implementation.

Model confidence: high.

Model note: The question asks for four access methods based on hardware implementation; the standard set from Stallings is sequential, direct, random, and associative. Some textbooks may classify SSD as random access or as a separate category; the answer uses the standard classification. No figure or table was supplied, so no limitation from extracted text applies.

QUESTION 9 (10 marks)

Source: CSC429-2019-06, page [3]

Unofficial model answer:

a) i) 10111.101_2 to decimal: Integer part $10111_2 = 1\times16 + 0\times8 + 1\times4 + 1\times2 + 1\times1 = 16+4+2+1 = 23$. Fractional part $.101_2 = 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} = 0.5 + 0 + 0.125 = 0.625$. Therefore $10111.101_2 = 23 + 0.625 = 23.625_{10}$. a) ii) 507.875_{10} to hexadecimal: Integer part: 507 ÷ 16 = 31 remainder 11 (B); 31 ÷ 16 = 1 remainder 15 (F); 1 ÷ 16 = 0 remainder 1. Reading remainders upward gives $1FB_{16}$. Fractional part: 0.875 × 16 = 14.0 → digit 14 (E), fractional remainder 0, so stop. Fraction = $.E_{16}$. Therefore $507.875_{10} = 1FB.E_{16}$. b) The operands are 988_{16} and $AB1_{16}$ (the 'O' in 'ABIO' is read as digit 1, since O is not a hexadecimal digit). In binary: 988_{16} = $1001\ 1000\ 1000_2$ and $AB1_{16} = 1010\ 1011\ 0001_2$, both 12-bit values. b) i) Sign and magnitude: Use 13-bit sign-and-magnitude (1 sign bit + 12 magnitude bits). $+988_{16}$ = 0 1001 1000 1000; $+AB1_{16}$ = 0 1010 1011 0001. To subtract, take the negative of $AB1_{16}$: $-AB1_{16}$ = 1 1010 1011 0001. Align and subtract magnitudes: $988_{16} - AB1_{16}$. Since $AB1_{16} > 988_{16}$, compute $AB1_{16} - 988_{16} = 129_{16}$ (check: 1010 1011 0001 − 1001 1000 1000 = 0001 0010 1001 = 129_{16}). The result is negative, so the sign bit is 1 and the magnitude is 129_{16}: result = 1 0001 0010 1001 in 13-bit sign-and-magnitude, i.e. -129_{16}. b) ii) 2's complement: Use 13-bit two's complement (extra bit for sign). $+988_{16}$ = 0 1001 1000 1000. $+AB1_{16}$ = 0 1010 1011 0001; its two's complement (negation) is found by inverting bits and adding 1: invert 0 1010 1011 0001 → 1 0101 0100 1110; add 1 → 1 0101 0100 1111. Now add: 0 1001 1000 1000 + 1 0101 0100 1111 = 1 1110 1101 0111 (discard any carry beyond 13 bits). This is a negative two's complement value; to read its magnitude, take two's complement again: invert 1 1110 1101 0111 → 0 0001 0010 1000; add 1 → 0 0001 0010 1001 = 129_{16}. Hence the result is -129_{16}, consistent with part (i).

Marking points:

© Hak Cipta Universiti Teknologi MARA

• a) i) 1 mark: correct integer conversion $10111_2 = 23_{10}$

• a) i) 1 mark: correct fractional conversion $.101_2 = 0.625_{10}$ and final answer 23.625_{10}

• a) ii) 1 mark: correct integer part $507_{10} = 1FB_{16}$ with division-by-16 work shown

• a) ii) 1 mark: correct fractional part $0.875_{10} = .E_{16}$ and final answer $1FB.E_{16}$

• b) 1 mark: correct interpretation of operands as 988_{16} and $AB1_{16}$ and correct binary/bit-width representation

• b) i) 1 mark: correct sign-and-magnitude representation of both operands (sign bit + magnitude)

• b) i) 1 mark: correct magnitude subtraction and correct signed result -129_{16} (sign bit 1)

• b) ii) 1 mark: correct two's complement (negation) of $AB1_{16}$

• b) ii) 1 mark: correct binary addition giving 1 1110 1101 0111

• b) ii) 1 mark: correct interpretation of result as -129_{16} (re-negation shown), matching part (i)

Model confidence: medium.

Model note: The question text contains 'ABIO'; since 'O' is not a hexadecimal digit, it is assumed to be the digit '1' ($AB1_{16}$), a common OCR/typing error. If the intended operand differs, the arithmetic changes. A 13-bit width (12 magnitude bits plus sign) is assumed so the sign bit is explicit; other widths give the same value but different bit patterns. No figure/table was supplied beyond the question text.

QUESTION 10 (10 marks)

Source: CSC429-2025-07, page [3]

Unofficial model answer:

a) The CPU consists of three main components: the control unit (CU), the arithmetic logic unit (ALU), and registers. The control unit fetches instructions from memory, decodes them, and generates the control signals that coordinate the operation of all other CPU components and the system bus; it acts as the 'manager' of the CPU. The ALU performs arithmetic operations (such as addition, subtraction, multiplication, and division) and logical operations (such as AND, OR, NOT, and comparisons) on data held in registers. Registers are small, high-speed storage locations inside the CPU that hold data, instructions, addresses, and status information during processing; examples include the program counter (PC), instruction register (IR), memory address register (MAR), memory data register (MDR), and accumulator (ACC). Together, these components execute the fetch-decode-execute cycle.

b) The machine cycle consists of fetch, decode, and execute phases, with the bus system (address bus, data bus, and control bus) carrying information between the CPU and memory/I/O. If the bus system is faulty, the machine cycle is disrupted. During the fetch phase, the CPU places the address of the next instruction on the address bus and asserts a read signal on the control bus; a faulty address bus may deliver a wrong address, so the wrong instruction or data is fetched, while a faulty data bus may corrupt the instruction or data being transferred. A faulty control bus may prevent the memory read/write signal from reaching memory, so no data is returned or the wrong operation is performed. Consequently, the instruction register may hold an invalid instruction, causing a decode error or an illegal opcode. In the execute phase, operands cannot be reliably read from or written to memory or I/O, leading to incorrect results, exceptions, or a system crash. Overall, the machine cycle cannot complete correctly, and the CPU may hang, produce erroneous results, or trigger a bus error exception.

Marking points:

• 1 mark: Identifies the control unit and explains its role in fetching, decoding, and generating control signals.

• 1 mark: Identifies the ALU and explains its role in performing arithmetic and logical operations.

• 1 mark: Identifies registers and explains their role as high-speed temporary storage for data, instructions, and addresses.

• 1 mark: Mentions specific registers (e.g., PC, IR, MAR, MDR, ACC) and their functions.

• 1 mark: Describes the fetch phase and the role of the address bus, data bus, and control bus.

• 1 mark: Explains the effect of a faulty address bus (wrong address fetched).

• 1 mark: Explains the effect of a faulty data bus (corrupted instruction/data).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Explains the effect of a faulty control bus (read/write signal not delivered).

• 1 mark: Describes the consequence for the decode phase (invalid instruction, illegal opcode).

• 1 mark: Describes the consequence for the execute phase (incorrect results, exception, system crash).

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify a particular CPU architecture or bus width, so the answer is kept general. The marking points assume a total of 10 marks, with 5 marks for part (a) and 5 marks for part (b).

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-07, page [5]

Unofficial model answer:

The equation in Figure 1 is read as z = (A² − B*C + jl_)/(D + 5). The OCR text is garbled ('jl_' and 'D+5' appear on a separate line), so the expression is interpreted as a numerator (A² − B*C + jl_) divided by the denominator (D + 5). Because the instruction set in Table 1 contains only ADD, SUB, MUL and DIV (no square or jump instructions), A² must be computed as A*A, and the term jl_ is treated as a single named operand/variable (call it J). Registers A, B, C, D are fixed; x, y, z are temporary, and the final result must end in z. The computation order is: t1 = A*A; t2 = B*C; t3 = t1 − t2; t4 = t3 + J; t5 = D + 5; z = t4 / t5.

(a) THREE-ADDRESS form. Each instruction names two sources and one destination, so no explicit temporaries in memory are needed beyond the named registers:

```
MUL (A, A, x)     ; x = A*A
MUL (B, C, y)     ; y = B*C
SUB (x, y, x)     ; x = A*A - B*C
ADD (x, J, x)     ; x = A*A - B*C + J
ADD (D, 5, y)     ; y = D + 5
DIV (x, y, z)     ; z = (A*A - B*C + J)/(D + 5)
```

This is the most compact form: six instructions, no PUSH/POP traffic.

(b) ONE-ADDRESS form. There is a single implicit accumulator; LOAD/STOR move between accumulator and memory, and ADD p/SUB p/MUL p/DIV p operate accumulator ← accumulator op p. Temporaries x and y hold intermediate values:

```
LOAD A        ; ACC = A
MUL A         ; ACC = A*A
STOR x        ; x = A*A
LOAD B        ; ACC = B
MUL C         ; ACC = B*C
STOR y        ; y = B*C
LOAD x        ; ACC = A*A
SUB y         ; ACC = A*A - B*C
ADD J         ; ACC = A*A - B*C + J
STOR x        ; x = numerator
LOAD D        ; ACC = D
ADD #5        ; ACC = D + 5   (immediate 5; if only memory operands allowed, STOR a constant 5 first)
STOR y        ; y = D + 5
LOAD x        ; ACC = numerator
DIV y         ; ACC = numerator/(D + 5)
STOR z        ; z = final answer
```

(c) ZERO-ADDRESS form. A stack machine: PUSH p pushes operand p, POP p pops top of stack into p, and ADD/SUB/MUL/DIV pop two operands and push the result (second-popped op top-popped). Postfix evaluation of (A*A − B*C + J)/(D + 5):

```
PUSH A
PUSH A
MUL           ; stack: A*A
PUSH B
PUSH C
```

© Hak Cipta Universiti Teknologi MARA

```
MUL         ; stack: A*A, B*C
SUB         ; stack: A*A - B*C
PUSH J
ADD         ; stack: A*A - B*C + J
PUSH D
PUSH 5
ADD         ; stack: numerator, D+5
DIV         ; stack: numerator/(D+5)
POP z       ; z = final answer
```

In all three versions the final result is stored in z, as required, and only the permitted instructions from Table 1 are used.

Marking points:

*• 2 marks: correct interpretation of Figure 1 as z = (A*A - B*C + J)/(D + 5), including handling A^2 as A*A and treating the garbled 'jl_' term as a single operand J (award partial if the denominator grouping is missed).*

• 5 marks: three-address program — correct use of MUL/SUB/ADD/DIV with (p,q,r) operand order, correct evaluation order, temporaries x and y used legally, final DIV writing to z (about 1 mark per correct instruction grouping).

• 6 marks: one-address program — correct LOAD/STOR/ADD p/SUB p/MUL p/DIV p sequence, accumulator used correctly, temporaries x and y for numerator and denominator, final STOR z (deduct for missing STOR or wrong operand order).

• 6 marks: zero-address program — correct postfix ordering with PUSH/POP/ADD/SUB/MUL/DIV, stack discipline correct (two pops, one push per arithmetic op), final POP z.

• 1 mark: overall correctness — all three programs compute the same expression and leave the answer in z, using only instructions listed in Table 1.

Model confidence: medium.

*Model note: Figure 1 is supplied only as a cropped image with garbled OCR text ('z=A2-B*C+jl_' and 'D+5'); the exact numerator term (appearing as 'jl_') and the division grouping are inferred. If the intended expression differs (e.g. the denominator applies only to part of the numerator, or 'jl_' is two operands j and l), the instruction sequences must be adjusted accordingly. The one-address immediate 'ADD #5' assumes an immediate form; if only memory operands are permitted, a constant 5 must be stored to a temporary first. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA